DS 642: Applications of Parallel Computing

Shahriar Afkhami

Week 8:
Introduction to OpenCL

What is OpenCL (Open Computing Language)

- OpenCL is a multi-vendor open standard for general-purpose parallel programming of heterogeneous systems including CPUs, GPUs, and other processors.
- Industry Standards for Programming Heterogeneous Platforms
    - OpenCL has enabled parallel computing in new markets: Mobile phones, cars, avionics
- OpenCL Platform Model
    - One Host and one or more OpenCL Devices
        - Each OpenCL Device is composed of one or more Compute Units
        - Each Compute Unit is divided into one or more Processing Elements
    - Memory divided into host memory and device memory

The BIG idea behind OpenCL

Replace loops with functions (a kernel) executing at each point in a problem domain

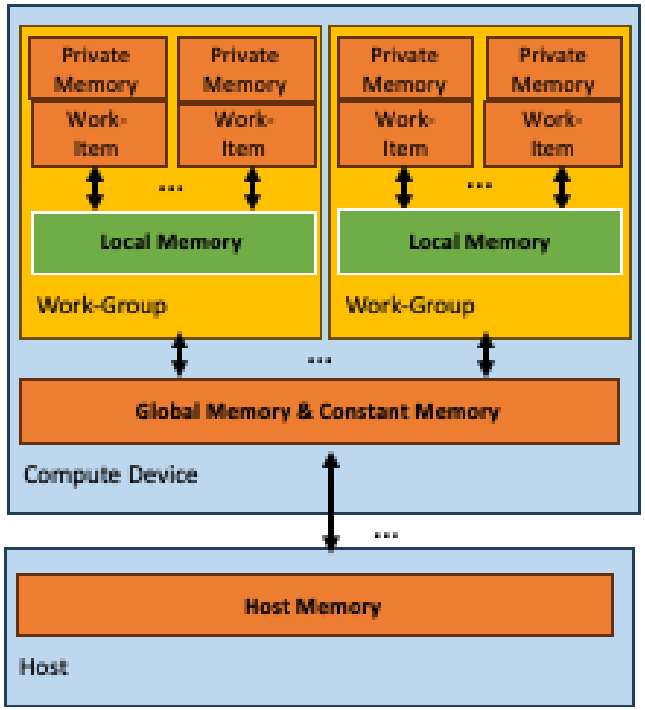
Traditional loops

```
void
mul(const  int n,
    const  float *a,
    const  float *b,
           float *c)
{
  int i;
  for (i = 0; i < n; i++)
    c[i] = a[i] * b[i];
}
```

OpenCL

```
__kernel void
mul(__global const float *a,
    __global const float *b,
    __global       float *c)
{
  int id = get_global_id(0);
  c[id]  = a[id] * b[id];
}
// execute over n work-items
```

OpenCL Memory model



Memory management is explicit: You are responsible for moving data from host -> global -> local and back

Context and Command-Queues

- Context: The environment within which kernels execute and in which synchronization and memory management is defined.
  - One or more devices
  - Device memory
  - One or more command-queues
- All commands for a device(kernel execution, synchronization, and memory operations) are submitted through a command-queue.
- Each command-queue points to a single device within a context.

Execution model (kernels)

- OpenCL execution model ... define a problem domain and execute an instance of a kernel for each point in the domain

```
1    // OpenCL kernel. Each work item takes care of one element of c
2
3    __kernel void addVectors(__global const float *a,
4                             __global const float *b,
5                             __global float *c) {
6
7            // Get our global thread ID
8            int id = get_global_id(0);
9
10           c[id] = a[id] + b[id];
11   }  // Execute over n work-items
```

Building Program Objects

- The program object encapsulates:
  - A context
  - The program source or binary
  - List of target devices and build options
- OpenCL uses runtime compilation, because in general the details of the target device is not known when porting the program
- The build process to create a program object:
  - Define source code for the kernel-program either as a string literal or read it from a file.
  - Create the program object and compile to create a “dynamic library” from which specific kernels can be pulled.

Example of vector addition

Khronos has defined a common C header file containing a high level interface to OpenCL, `cl.h`

The Host Code:

```
1  #include <CL/cl.h> // openCL headers (Khronos C Wrapper API)
2  #include <stdio.h>
3  #include <stdlib.h>
4
5
6  #define MAX_SOURCE_SIZE (0x100000)
7
8  int main(int argc, char ** argv) {
9
10   int SIZE = 1024;
11
12   // Allocate memories for input arrays and output array.
13   float *A = (float*)malloc(sizeof(float)*SIZE);
14   float *B = (float*)malloc(sizeof(float)*SIZE);
15
16   // Output
17   float *C = (float*)malloc(sizeof(float)*SIZE);
18
19   // Initialize values for array members.
20   for (int i = 0; i < SIZE; ++i) {
21     A[i] = i;
22     B[i] = i*2;
23   }
```

Example of vector addition

A kernel can be called from the host like a regular function:

```
1  // Load kernel from file vecAddKernel.cl
2
3  FILE *kernelFile;
4  char *kernelSource;
5  size_t kernelSize;
6
7  kernelFile = fopen("vecAddKernel.cl", "r");
8
9  if (!kernelFile) {
10   fprintf(stderr, "No file named vecAddKernel.cl was found\n");
11   exit(-1);
12 }
13
14 kernelSource = (char*)malloc(MAX_SOURCE_SIZE);
15 kernelSize = fread(kernelSource, 1, MAX_SOURCE_SIZE, kernelFile);
16 fclose(kernelFile);
17
18 // Getting platform and device information
19 cl_platform_id platformId = NULL; // OpenCL platform - request a device
20 cl_device_id deviceID = NULL;     // Device ID
21 cl_uint retNumDevices;
22 cl_uint retNumPlatforms;
23 cl_int ret = clGetPlatformIDs(1, &platformId, &retNumPlatforms);   // Bind to
     platform
24 ret = clGetDeviceIDs(platformId, CL_DEVICE_TYPE_DEFAULT, 1, &deviceID, &
     retNumDevices); // Get ID for the device
```

Example of vector addition

```
1  // Creating context
2  cl_context context = clCreateContext(NULL, 1, &deviceID, NULL, NULL,  &ret);
3
4  // Creating command queue - sequence of commands sent to device
5  cl_command_queue queue = clCreateCommandQueueWithProperties(context, deviceID, 0, &
       ret);
6
7  // Create the input and output arrays in device memory for our calculation
8  // Memory buffers for each array
9  cl_mem d_a = clCreateBuffer(context, CL_MEM_READ_ONLY, SIZE * sizeof(float), NULL,
       &ret);
10 cl_mem d_b = clCreateBuffer(context, CL_MEM_READ_ONLY, SIZE * sizeof(float), NULL,
       &ret);
11 cl_mem d_c = clCreateBuffer(context, CL_MEM_WRITE_ONLY, SIZE * sizeof(float), NULL,
        &ret);
12
13 // Copy lists to memory buffers
14 ret = clEnqueueWriteBuffer(queue, d_a, CL_TRUE, 0, SIZE * sizeof(float), A, 0, NULL
       , NULL);;
15 ret = clEnqueueWriteBuffer(queue, d_b, CL_TRUE, 0, SIZE * sizeof(float), B, 0, NULL
       , NULL);
16
17 // Create program from kernel source buffer
18 cl_program program = clCreateProgramWithSource(context, 1, (const char **)&
       kernelSource, (const size_t *)&kernelSize, &ret);
19
20 // Build program executable (in OpenCL the kernel is compiled at runtime)
21 ret = clBuildProgram(program, 1, &deviceID, NULL, NULL, NULL);
```

Example of vector addition

```
1  // Create the compute kernel (compile the kernel code)
2  cl_kernel kernel = clCreateKernel(program, "addVectors", &ret);
3
4  // Set arguments for kernel function
5  ret = clSetKernelArg(kernel, 0, sizeof(cl_mem), (void *)&d_a);
6  ret = clSetKernelArg(kernel, 1, sizeof(cl_mem), (void *)&d_b);
7  ret = clSetKernelArg(kernel, 2, sizeof(cl_mem), (void *)&d_c);
8
9  // Execute the kernel
10 size_t globalItemSize = SIZE; // Number of work items in each local work group
11 size_t localItemSize = 64; // Number of total work items - globalItemSize has to be
       a multiple of localItemSize. 1024/64 = 16
12 ret = clEnqueueNDRangeKernel(queue, kernel, 1, NULL, &globalItemSize, &
       localItemSize, 0, NULL, NULL); // Execute the kernel over the entire range of
       the data set
13
14 // Wait for the command queue to get serviced before reading back results
15 clFinish(queue);
16
17 // Read from device back to host
18 ret = clEnqueueReadBuffer(queue, d_c, CL_TRUE, 0, SIZE * sizeof(float), C, 0, NULL,
       NULL);
```

Example of vector addition

```
1   // Check the results
2   for (i = 0; i < SIZE; ++i)
3     if (C[i] != (A[i] + B[i])) {
4       printf("FAILED!. \n");
5       break;
6     }
7   printf("PASSED! \n");
8
9
10  // Clean up, free the memory.
11  ret = clFlush(queue);
12  ret = clFinish(queue);
13  ret = clReleaseCommandQueue(queue);
14  ret = clReleaseKernel(kernel);
15  ret = clReleaseProgram(program);
16  ret = clReleaseMemObject(d_a);
17  ret = clReleaseMemObject(d_b);
18  ret = clReleaseMemObject(d_c);
19  ret = clReleaseContext(context);
20  free(A);
21  free(B);
22  free(C);
23
24  return 0;
25 }
```

Examples of vector addition in parallel

The Device Code:

```
1  // OpenCL kernel. Each work item takes care of one element of c
2
3  __kernel void addVectors(__global const float *a,
4                           __global const float *b,
5                           __global float *c) {
6
7      // Get our global thread ID
8      int id = get_global_id(0);
9
10     c[id] = a[id] + b[id];
11 }  // Execute over n work-items
```

The Kernel Invocation:

Source code for the kernel-program is read from a file.

Compiling

On Bridges-2, GPU nodes are compute capability 70 (Volta and Tesla):

```
$ module load cuda
$ nvcc vector_add1.c -o vector_add -lm -lOpenCL \
-arch=compute_70 -code=sm_70
```

OpenCL to CUDA

- Mapping Data Parallelism Models:

OpenCL Parallelism Concept	CUDA Equivalent
kernel	kernel
host program	host program
NDRange (index space)	grid
work item	thread
work group	block

OpenCL to CUDA

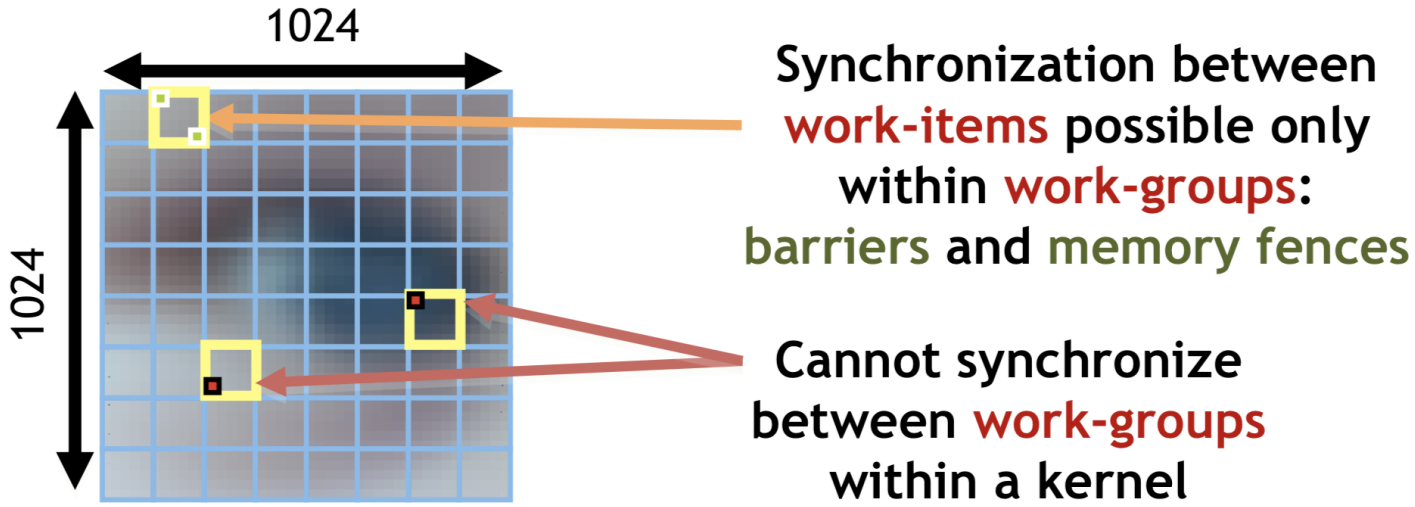
- Mapping indices

OpenCL API call	Explanation	CUDA equivalent
get_global_id(0);	Global index of the work item in the *x*-dimension	blockIdx.x×blockDim.x+threadIdx.x
get_local_id(0)	Local index of the work item within the work group in the *x*-dimension	threadIdx.x
get_global_size(0);	Size of NDRange in the *x*-dimension	gridDim.x ×blockDim.x
get_local_size(0);	Size of each work group in the *x*-dimension	blockDim.x

An N-dimensional domain of work-items

- Kernels run over global dimension index ranges (NDRange), broken up into “work groups”, and “work items”:

- **Global** Dimensions:
  - 1024x1024 (whole problem space)
- **Local** Dimensions:
  - 128x128 (**work-group**, executes together)



An N-dimensional domain of work-items

- Kernels run over global dimension index ranges (NDRange), broken up into “work groups”, and “work items”.
- Work items executing within the same work group can synchronize with each other using barriers or memory fences.
- Work items in different work groups can only sync with each other by launching a new kernel.

OpenCL Steps

1. Obtain OpenCL platform (getting the platform ID)
2. Obtain device ID for at least one device (accelerator)
3. Create context for device
4. Create accelerator program from source code
5. Build the program
6. Create kernel(s) from program functions
7. Create command queue for target device
8. Allocate device memory / move input data to device memory
9. Associate arguments to kernel with kernel object
10. Deploy kernel for device execution
11. Move output data to host memory
12. Release context/program/kernels/memory

OpenCL Steps

The host program is the code that runs on the host to:

- Setup the environment for the OpenCL program
- Create and manage kernels

1. Obtain OpenCL platform (getting the platform ID)

```
1 cl_int ret =
2 clGetPlatformIDs(1, &platformId, &retNumPlatforms);
```

platform = devices + context + queues

OpenCL Steps

2- Use platform to get ID for device

```
1 ret =
2 clGetDeviceIDs(platformId, CL_DEVICE_TYPE_DEFAULT, 1,
3                 &deviceID, &retNumDevices);
```

Device is requested from the platform using `clGetDeviceIDs`

OpenCL Steps

4- Create accelerator program from source code

`kernelSource`

is a string, either statically set in the host program or returned from a function that loads the kernel code from a file.

5- OpenCL accelerator code is compiled at run-time

`clBuildProgram`

6- Create cl_kernel(s) from program functions

`clCreateKernel`

Create the program object and compile to create a “dynamic library” from which specific kernels can be pulled.

OpenCL Steps

7- Create command queue for kernel dispatch

- Commands include: Kernel executions, Memory object management, Synchronization
- The only way to submit commands to a device is through a command-queue.
- Each command-queue points to a single device within a context.
- Multiple command-queues can feed a single device: Used to define independent streams of commands that don't require synchronization.

```
1 cl_command_queue queue =
2 clCreateCommandQueueWithProperties(context, deviceID, 0, &ret)
```

Can be in-order or out-of-order execution and guaranteed to be completed at synchronization points.

OpenCL Steps

8- Allocate device memory / move input data to device

```
1 cl_mem d_a =
2 clCreateBuffer(context, CL_MEM_READ_ONLY,
3                SIZE * sizeof(float), NULL, &ret);
4 ret =
5 clEnqueueWriteBuffer(queue, d_a, CL_TRUE, 0,
6                       SIZE * sizeof(float), A, 0, NULL, NULL);
```

Create the device-side buffer, assign read- only memory to hold the host array and copy it into device memory.

OpenCL Steps

9- Associate arguments to kernel with kernel object

```
ret = clSetKernelArg(kernel, 0, sizeof(cl_mem), (void *)&d_a);
```

10- For kernel launches, specify global and local dimensions:

```
size_t globalItemSize = SIZE;
size_t localItemSize = 64;
```

11- Enqueue the kernel for device execution (can be CPU, GPU, or other accelerator)

```
ret =
clEnqueueNDRangeKernel(queue, kernel, 1, NULL,
                       &globalItemSize,
                       &localItemSize, 0, NULL, NULL);
```

OpenCL Steps

12- Write output device data back to host

```
1 ret =
2 clEnqueueReadBuffer(queue, d_c, CL_TRUE, 0,
3                     SIZE * sizeof(float), C, 0, NULL, NULL);
```

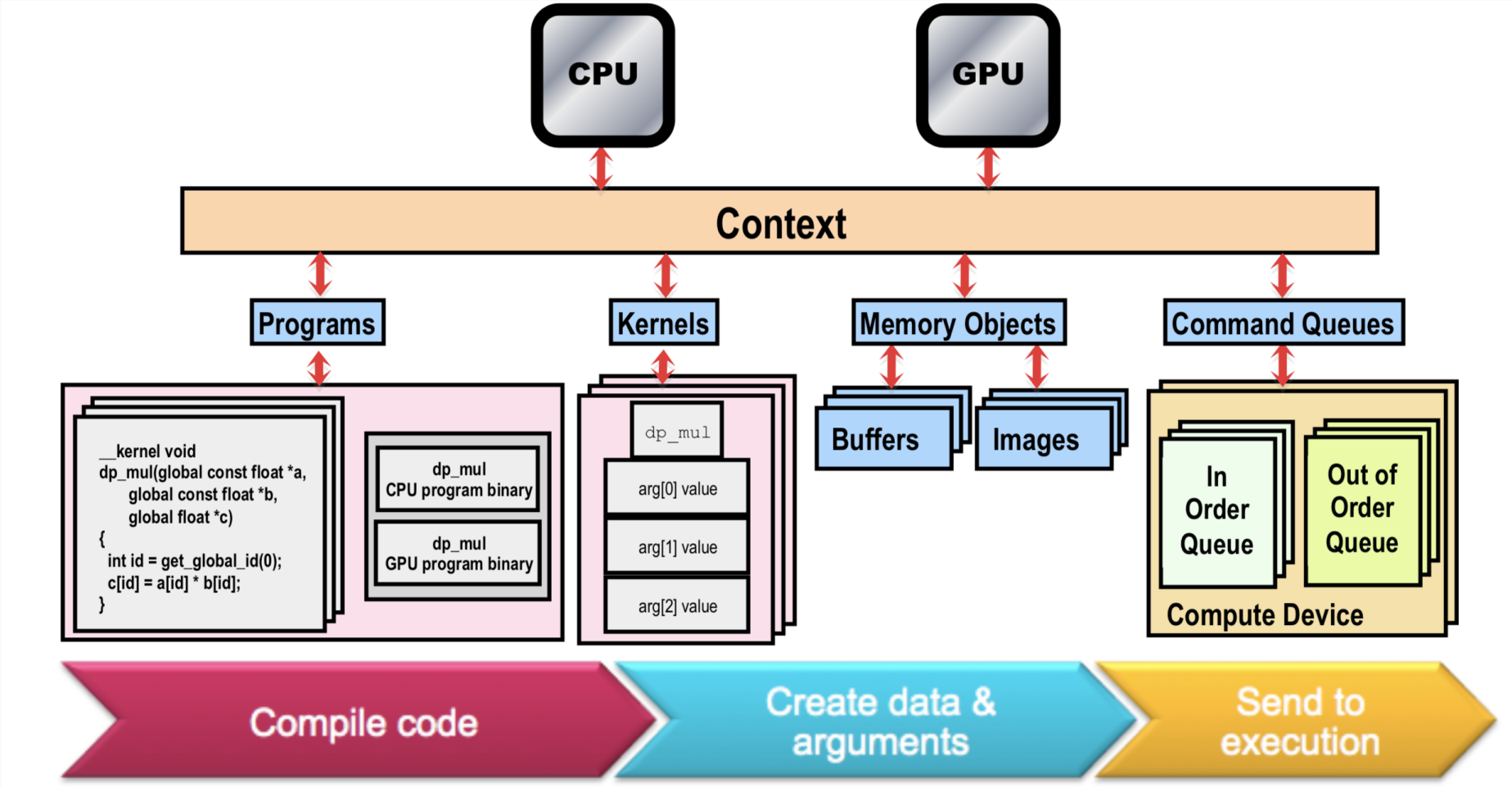
13- Release/clean context/program/kernels/memory

```
1 clReleaseCommandQueue(queue);
2 clReleaseKernel(kernel);
3 clReleaseProgram(program);
4 clReleaseMemObject(d_a);
5 clReleaseContext(context);
```

CUDA-OpenCL correspondence

Thread	⟷	Work-item
Thread-block	⟷	Work-group
Global memory	⟷	Global memory
Constant memory	⟷	Constant memory
Shared memory	⟷	Local memory
Local memory	⟷	Private memory
__global__ function	⟷	__kernel function
__device__ function	⟷	no qualification needed
__constant__ variable	⟷	__constant variable
__device__ variable	⟷	__global variable
__shared__ variable	⟷	__local variable

OpenCL Summary



https://www.khronos.org/files/opencl-1-2-quick-reference-card.pdf